

Курсовой проект

Тема проекта: "Музыкальный плеер с удаленной библиотекой и возможностью хранения связанных нот, табулатур"

Автор: Студент группы ВБД-321

Васильев Олег Олегович

Нижний Новгород, 2026

Введение

Основная задача курсовой работы закрепить на практике изученные языки программирования и технологии. Вторичная цель создать полезное приложение для обучающихся музыке.

Музыкальных сервисов для прослушивания музыки довольно много. Сервисов для обучения музыке, тоже сотни. Но профессиональной коллекции музыкальных файлов и связанных с ними нот почему-то нет. Весьма удобно, обучаясь игре на музыкальном инструменте не тратить время на поиск, а просто слушая произведение сразу посмотреть ноты. Поставить на повтор и попробовать сыграть вместе с воспроизведенным отрывком.

А для музыкантов играющих в ансамбле или поющих в хоре важно, чтобы все заранее послушали одно и тоже произведение, а не какие-то разные его версии.

Именно поэтому тема данного проекта: “Музыкальный плеер с удаленной библиотекой и возможностью хранения связанных нот, табулатур”.

1. Краткий обзор решения

Решение состоит из трех компонентов:

1. Серверное приложение ASP.NET 9.0 на базе шаблона Core Web API
2. База данных под управлением СУБД PostgreSQL
3. Клиентское десктопное приложение на JAVA FX.

Серверное приложение подключается к серверу базы данных и взаимодействует с ней посредством Microsoft Entity Framework Core. Клиентское приложение обращается к серверу посредством HTTP запросов.

В рамках курсового проекта реализована архитектура решения, и простейшая логика работы с музыкальными файлами и нотами. Получение информации с сервера, скачивание, прослушивание музыки, просмотр и скачивание дополнительных файлов.

Примечание: Авторизация, личные кабинеты пользователей, списки воспроизведения, права доступа, плейлисты могут быть добавлены в рамках дипломной работы.

2. Серверная часть

Хотелось сделать сервер независимым от платформ клиентских приложений.

Поэтому API реализован в виде GET и POST запросов. Клиентами могут быть: сайты, мобильные приложения, десктопные приложения, а также сторонние серверные приложения.

Проект организован по упрощённой версии шаблона "Слойная архитектура" (Layered Architecture), характерной для ASP.NET Core приложений. Основные слои можно выделить по структуре каталогов:

Объекты передачи данных (DTO/) - объекты для обмена по HTTP

Контроллеры (Controllers/) - принимают HTTP-запросы, делегируют логику службам и возвращают ответы.

Службы (Services/) - служат связующим звеном между контроллерами и данными (UploadService, MusicService, ExtraFileService)

Модели (Models/) - представляют основные сущности приложения - Музыкальный файл, Дополнительный файл.

Контекст данных (Database/) - включает ApplicationDbContext, который отвечает за взаимодействие с базой данных через Entity Framework Core.

Внедрение зависимостей ASP.NET Core

За внедрение зависимостей отвечает встроенный контейнер зависимостей (DIContainer) самого фреймворка ASP.NET Core. Он является частью Microsoft.Extensions.DependencyInjection и настраивается в Program.cs через объект builder.Services (тип IServiceCollection). Регистрация зависимостей в контейнер выполняется в [Program.cs](#). Внутри классов внедрение происходит автоматически по аргументам конструктора.

Т.к теоретически кол-во поступающих запросов на сервер может быть очень большим важно организовать безопасную многопоточную обработку. Тип жизненного цикла объектов определяется архитектурным шаблоном.

Контроллеры по умолчанию создаются как Transient (новый экземпляр на каждый запрос).

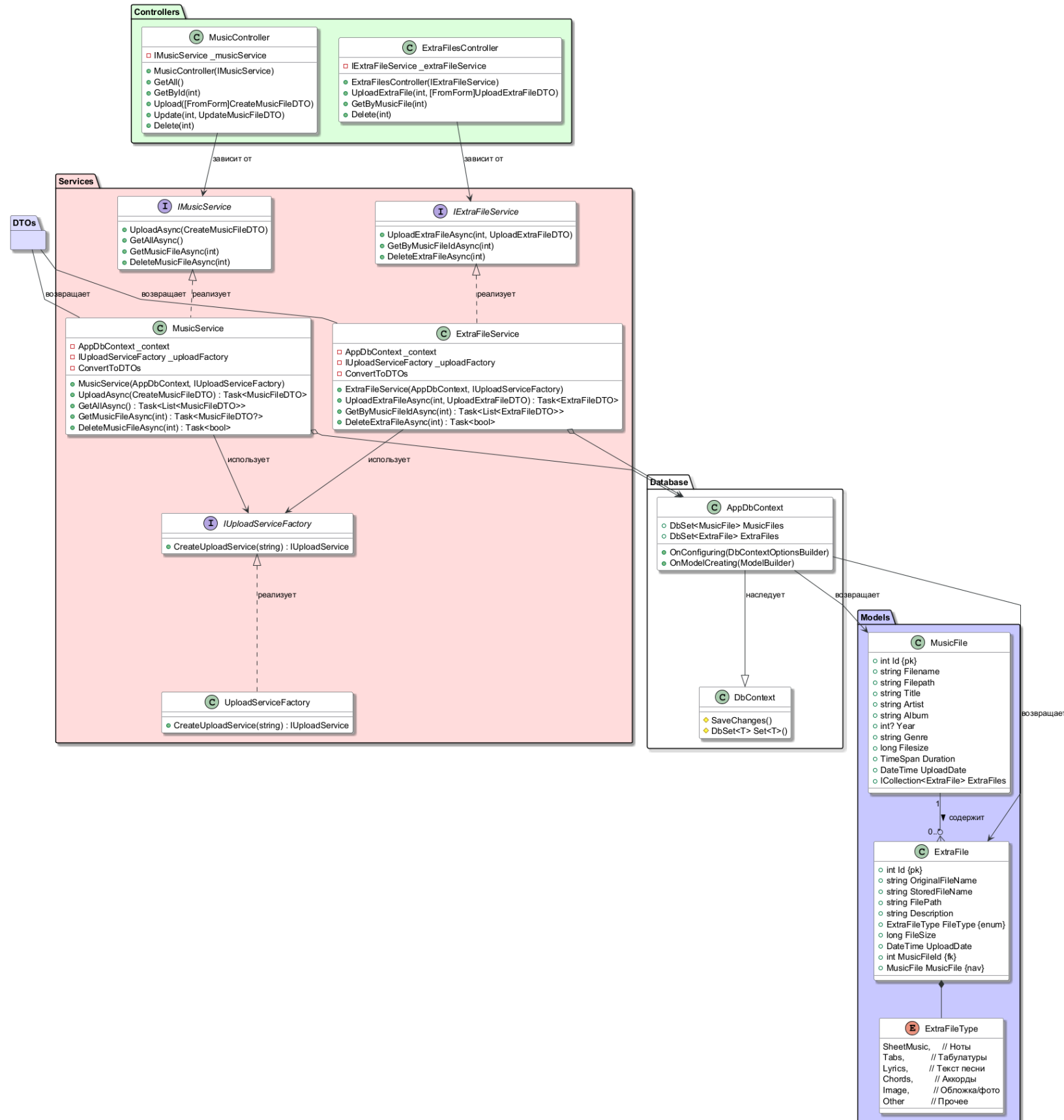
Пользовательские сервисы (MusicService, ExtraFileService) - Scoped один раз на каждый HTTP-запрос.

AppDbContext - Scoped. Это стандартный и критически важный выбор. DbContext должен жить в рамках одного HTTP-запроса. Это обеспечивает изоляцию единицы работы, корректное управление подключениями к БД и предотвращает состояние гонки при параллельных запросах.

IUploadServiceFactory - Фабрика, как правило, не содержит состояния, специфичного для запроса, и не зависит от DbContext. Её задача — создавать другие объекты. Регистрация как Singleton эффективна: экземпляр создаётся один раз при старте приложения и используется всеми запросами, что снижает накладные расходы.

Диаграмма классов серверного приложения

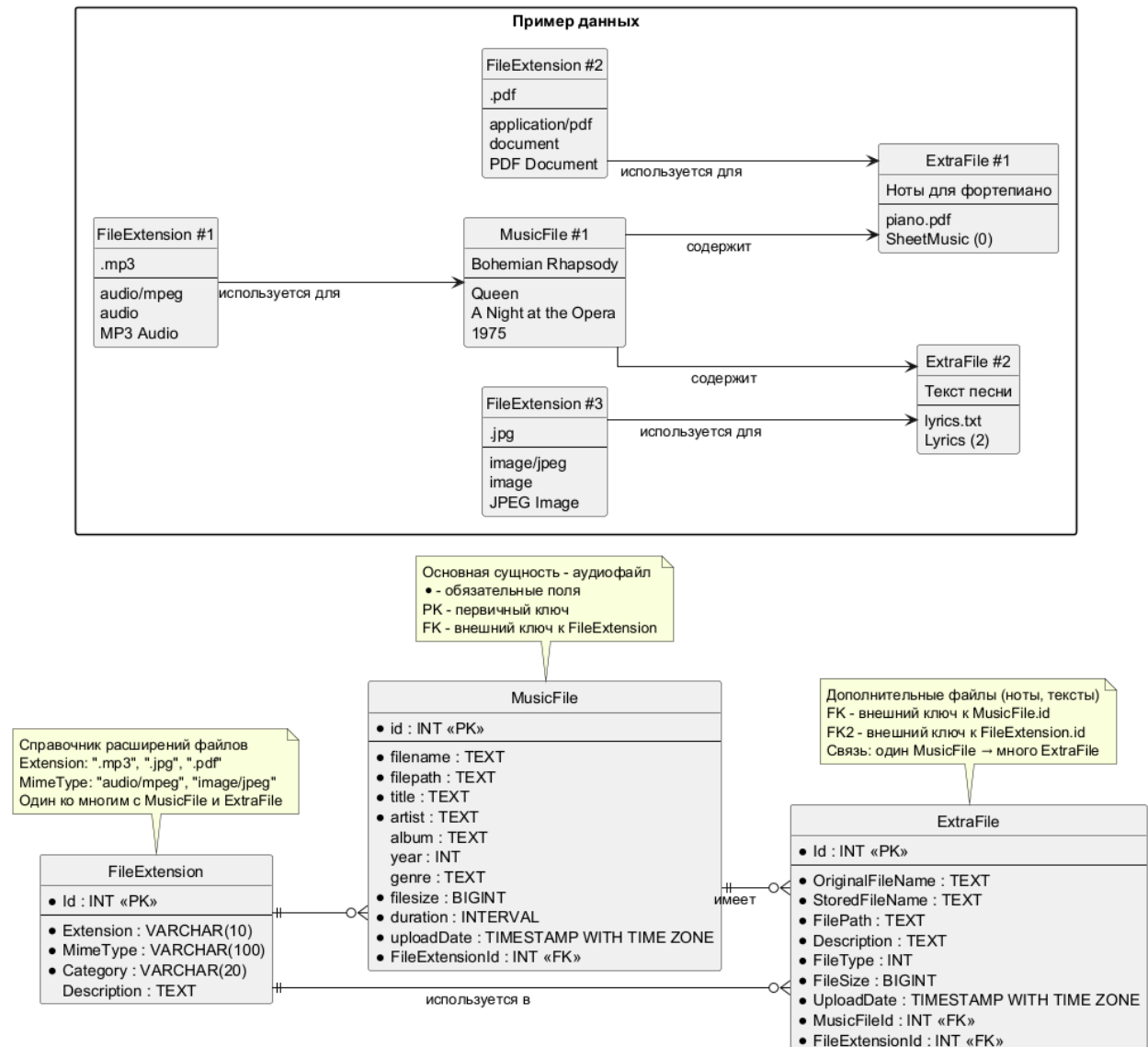
Полноразмерная диаграмма доступна по ссылке: <https://raw.githubusercontent.com/vaClev/>



3. Структура базы данных

Сущности (MusicFile, ExtraFile) требуют реляционной структуры с чёткими связями. Важна поддержка каскадного удаления (удалил трек — автоматически удалились связанные ноты). Также пользователю часто необходимо легко найти одну сущность по известной другой.

Фактически с этой задачей может справиться любая современная СУБД. PostgreSQL выбран для учебного проекта, т.к он свободный и открытый, и простой в развертывании. При развитии проекта для оптимизации поиска и нормализации планирую ввести дополнительные таблицы: Artists, Genres.



4. Клиентская часть

Клиентское приложение для компьютера хотелось сделать независимым от операционной системы, поэтому в качестве платформы выбрана JAVA FX.

Для запуска приложения необходимо:

- JAVA OpenJDK 21U
- JAVA FX version 21.0.9

Архитектура клиентского-приложения, также как и серверного, разделена на слои:

Объекты передачи данных (dto/) - объекты для обмена по HTTP. Объекты парсинга JSON ответов сервера.

Модели (Models/) - представляют основные сущности приложения - Музыкальный файл, Дополнительный файл.

Службы (services/) - по сути выполняют всю работу, возвращают результат контроллерам. (DownloadService, BasicPlayerService, MusicClientService)

Контроллеры (controllers/) - контроллеры UI элементов

UI Формы (views/) - FXML файлы разметки для интерфейса.

Внедрение зависимостей

Внедрение зависимостей в клиентском приложении выполнено вручную, т.к жизненный цикл сервисов довольно прост. Сервисы создаются в MusicProApplication::startServices, и внедряются в контроллер главного окна приложения. Все сервисы существуют в единственном экземпляре во время работы приложения(Singleton scope). После закрытия главного окна приложения сервисы завершают работу MusicProApplication::shutdownServices.

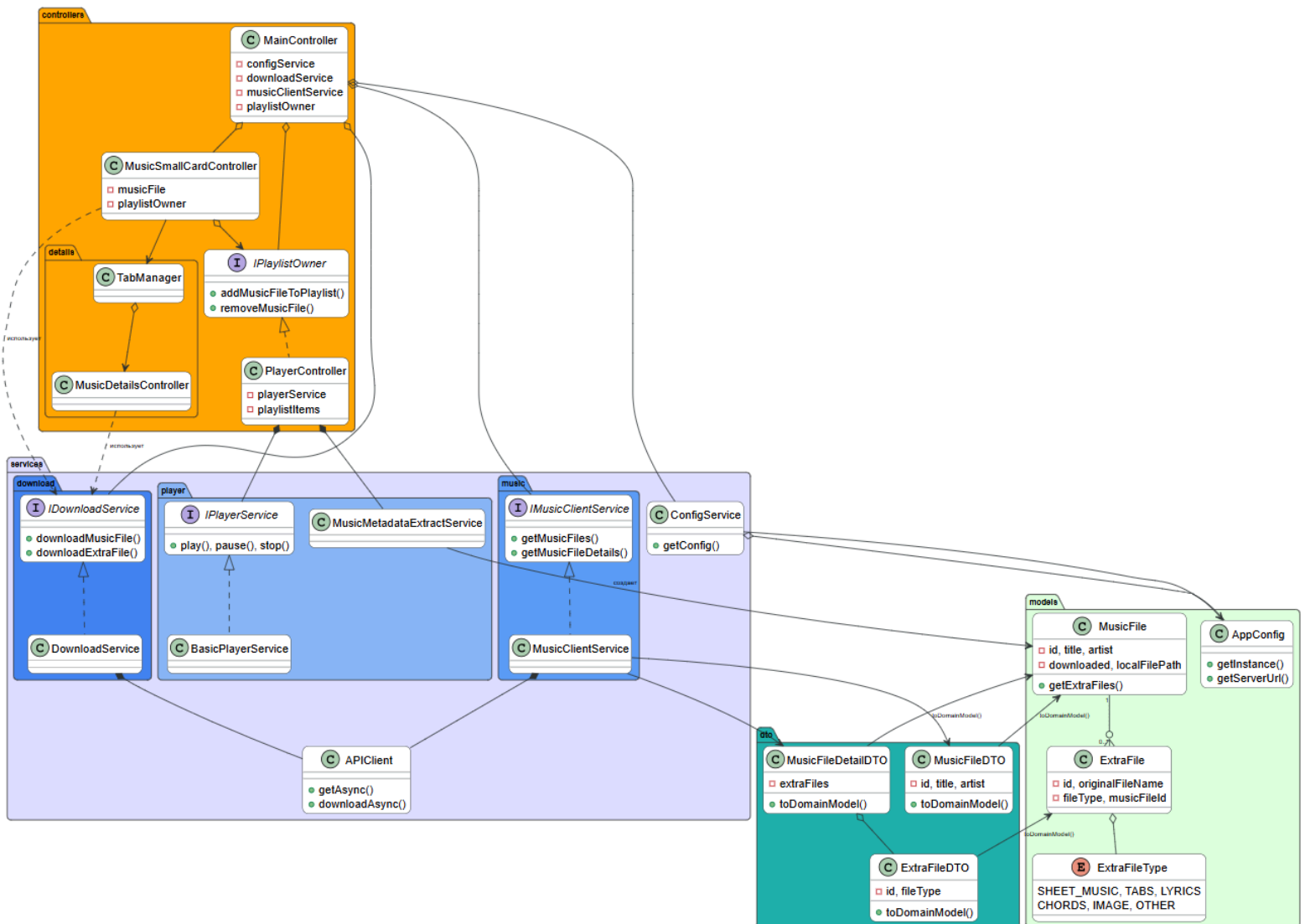
MainController предоставляет ссылки на сервисы своим дочерним компонентам, в частности карточкам песен, контроллеру плеера и менеджеру вкладок.

Примечание: ручное внедрение зависимостей приводит к нарушению некоторыми классами принципа единственной ответственности. В случае дальнейшего развития проекта, разумно перейти на Spring Framework.

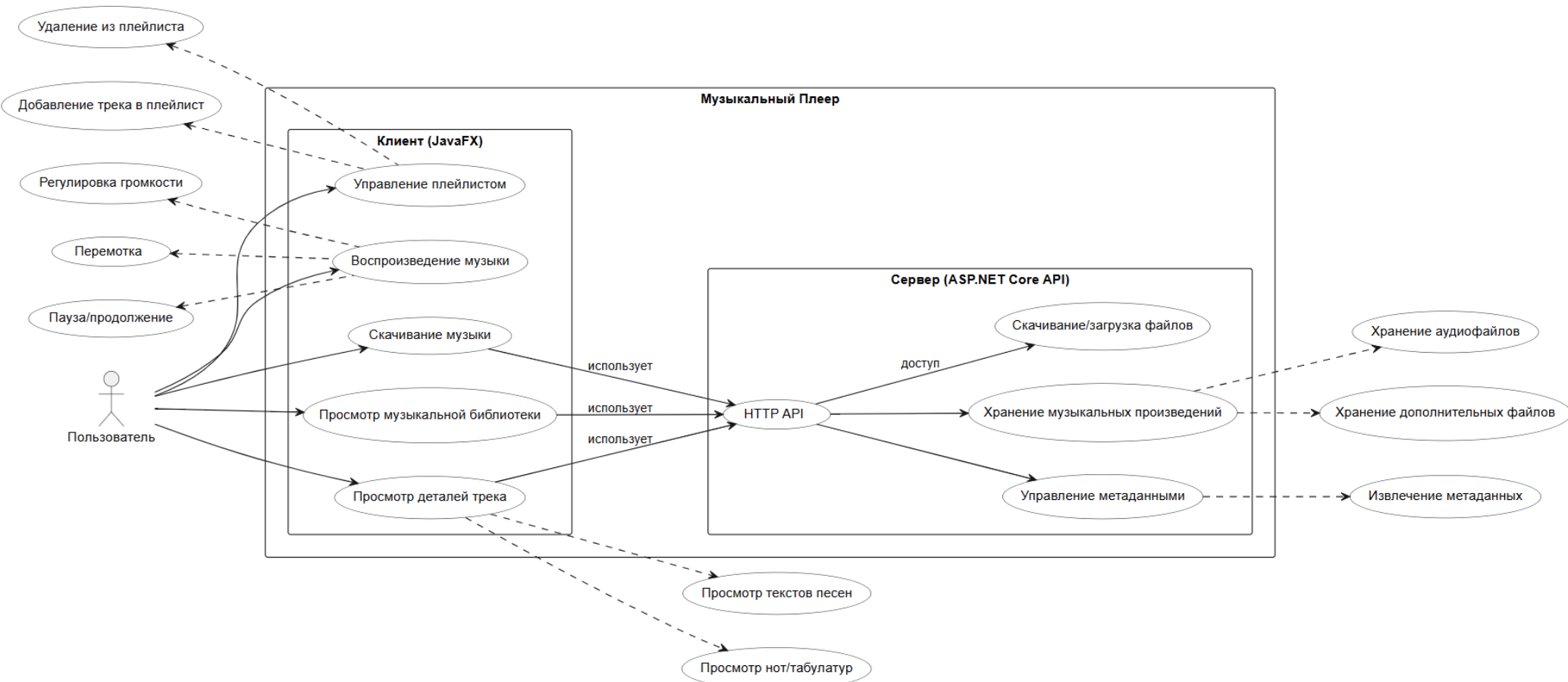
Диаграмма классов клиентского приложения

Полноразмерная диаграмма классов доступна по ссылке:

<https://raw.githubusercontent.com/vaClev/MusicPlayer/...../ClientClassDiagram.svg>



5. Диаграмма вариантов использования



6. Исходный код приложений

Исходный код доступен по ссылке: <https://github.com/vaClev/MusicPlayer>

Заключение

Для меня основной целью курсовой работы было изучить архитектуру клиент-серверных приложений и создать прототип приложения. Эта цель достигнута.

Итогом работы является простое с точки зрения конечного пользователя приложение, но его архитектура открыта для расширения и наращивания функциональности.

Серверное приложение, может быть легко адаптировано под хранение и скачивание любых файлов, например текстовых, графических документов, фото.